Arrays, Multidimensional Arrays & Pointers

Q. Define Arrays. Explain one-dimensional arrays, multi-dimensional arrays, and the concept of an array of pointers with syntax and examples.

> **Definition to Remember**

> An
**conti

---

1. One-Dimensional (1D) Array

- Simplest form a single list of elements.

- Use

```c
int arr[
```

2. Multi-Dimensional Arrays

- An array of arrays. Most common: **2D Array** (matrix rows and columns).

- Use

```c
int mat
```

```
Memo
```

3. Array of Pointers

- An array where each element is a **pointer** (holds a memory address).

- **S

```c
char *
```

4. Key Properties of Arrays

| Property | Detail |
| :--- | :

> **Must-Write Points (for 10 marks)**

> 1. Ar

> **Quick Recall**

> `Arra
access

Algorithmic Implementations: 2D Arrays & Sparse Matrix (C Code)

1. Matrix Multiplication Algorithm & C Code

```c
#includ
#define R1 2
#defin

void multiplyMatrices(int A[R1][C1], int B[R2][C2], int C[R1][C2]) {
for (int

int main() {
int A[2
```

multiplyMatrices(A, B, C);

printf("--- Resultant Product Matrix (2x2) ---\n");

for (int

---

2. Sparse Matrix Triplet Representation (C Code)

```c

#inclu

struct Element {

int row

struct Sparse {

int m;

void createSparse(struct Sparse *s, int matrix[4][4]) {

s->m =

for (int i = 0; i < 4; i++) {

for (int

void displaySparse(struct Sparse s) {

printf("
```

```
int main() {
int mat
struct Sparse s;
create
printf("--- Sparse Matrix Triplet Representation ---\n");
display
return 0;
}
```